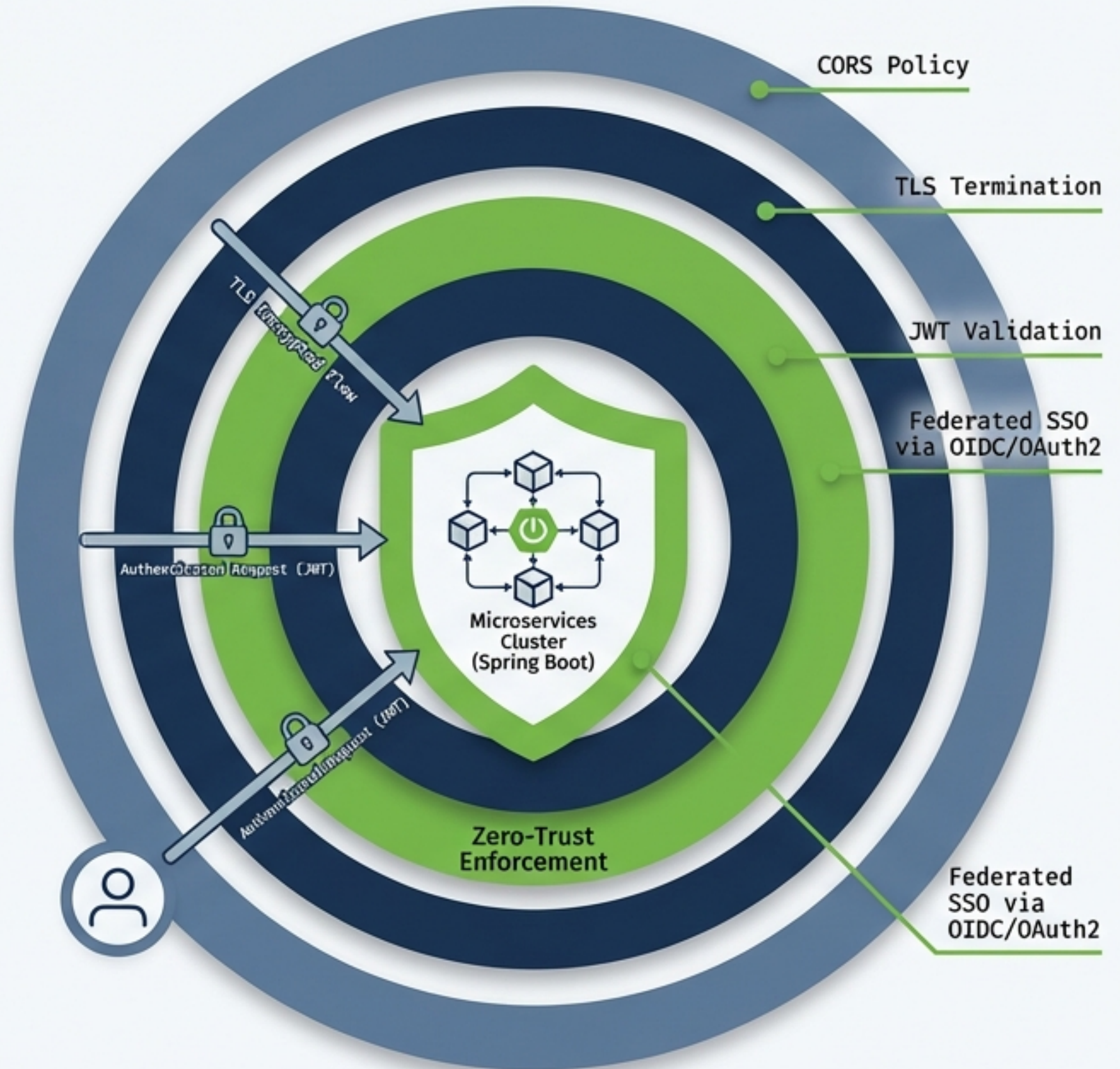


Seguridad Avanzada en Microservicios con Spring Boot

Arquitectura Zero-Trust: Desde CORS y TLS hasta JWT y SSO federado.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .cors().and()
            .csrf().disable()
            .authorizeRequests()
                .antMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .oauth2ResourceServer()
                .jwt();
    }
}
```



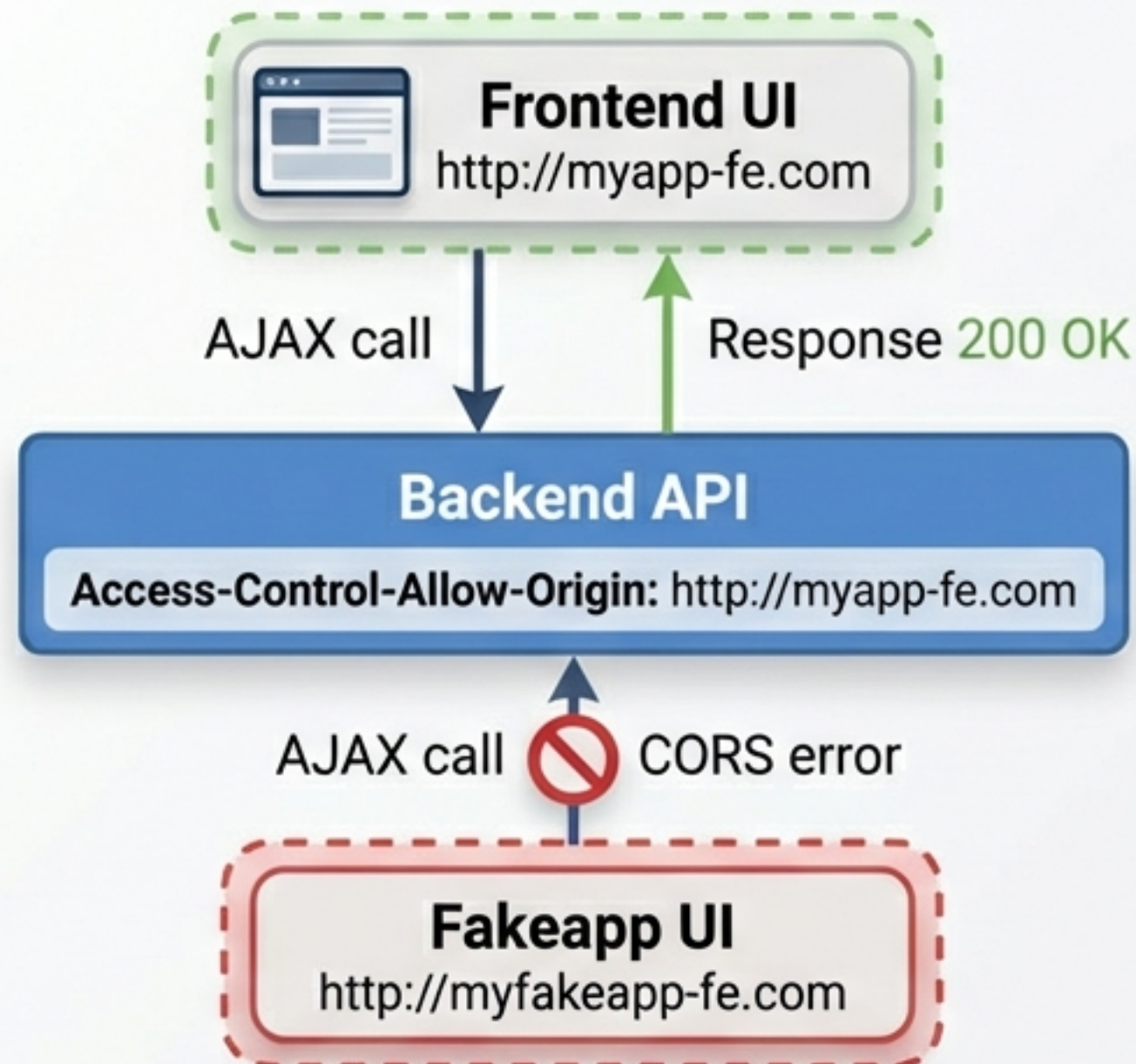
El Viaje de una Petición Segura (Filosofía Zero-Trust)

El paradigma ha cambiado: ya no basta con proteger el perímetro de la red. En una arquitectura de microservicios real, asumimos que la red interna ya está comprometida ("**Zero-Trust**"). **Nunca confíes, siempre verifica.**



Capa 1: Asegurando el Perímetro con CORS

Cross-Origin Resource Sharing (CORS) es la primera línea de defensa en el navegador, previniendo que dominios maliciosos realicen llamadas no autorizadas a tu API.



En Quarkus (application.properties):

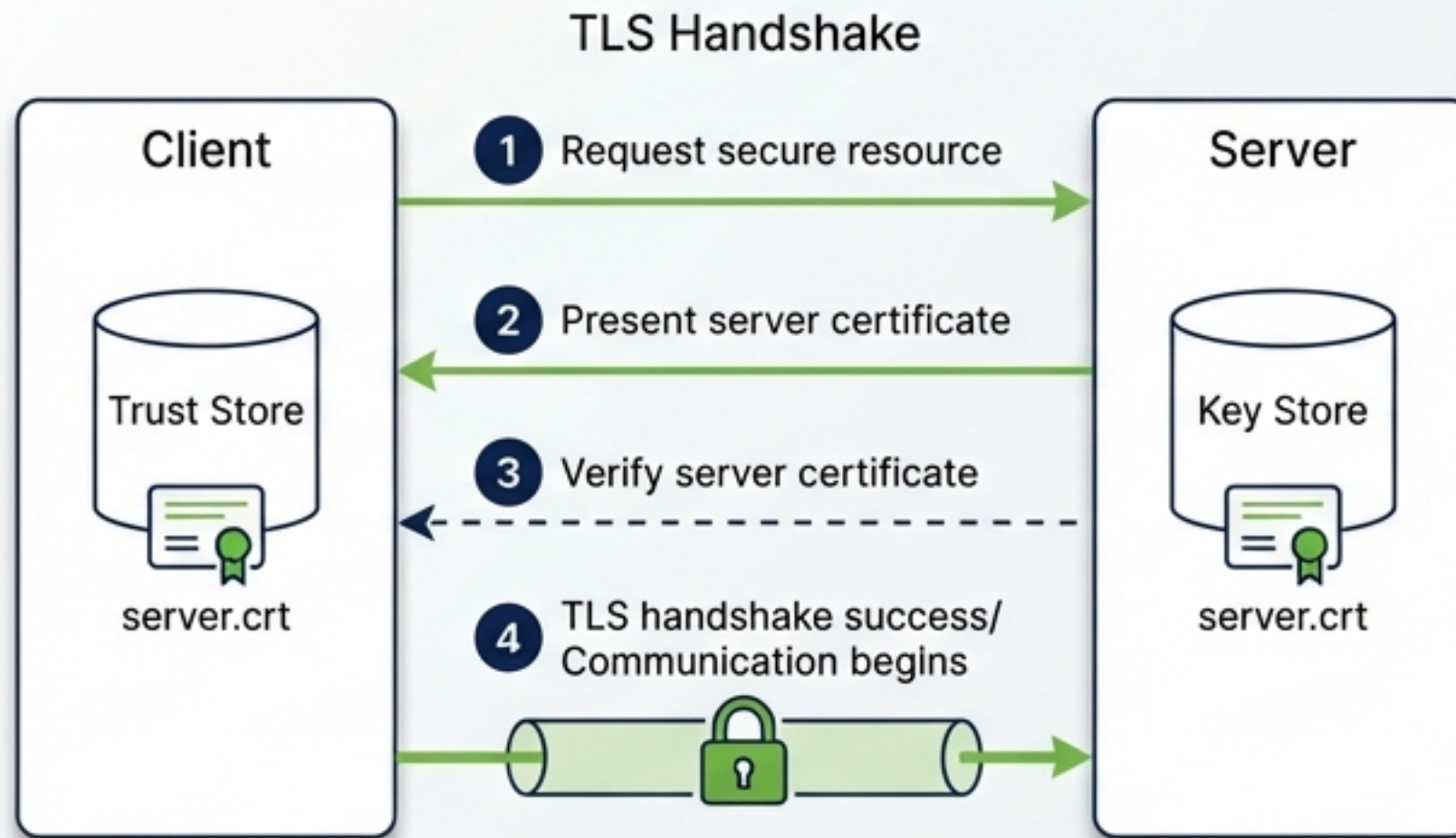
```
quarkus.http.cors=true
```

En Spring Boot (Java Config):

```
@CrossOrigin(origins = "http://myapp-fe.com")  
// Nivel Global vía WebMvcConfigurer  
public void addCorsMappings(CorsRegistry registry) { ... }
```


Capa 2: Encriptando el Canal con TLS

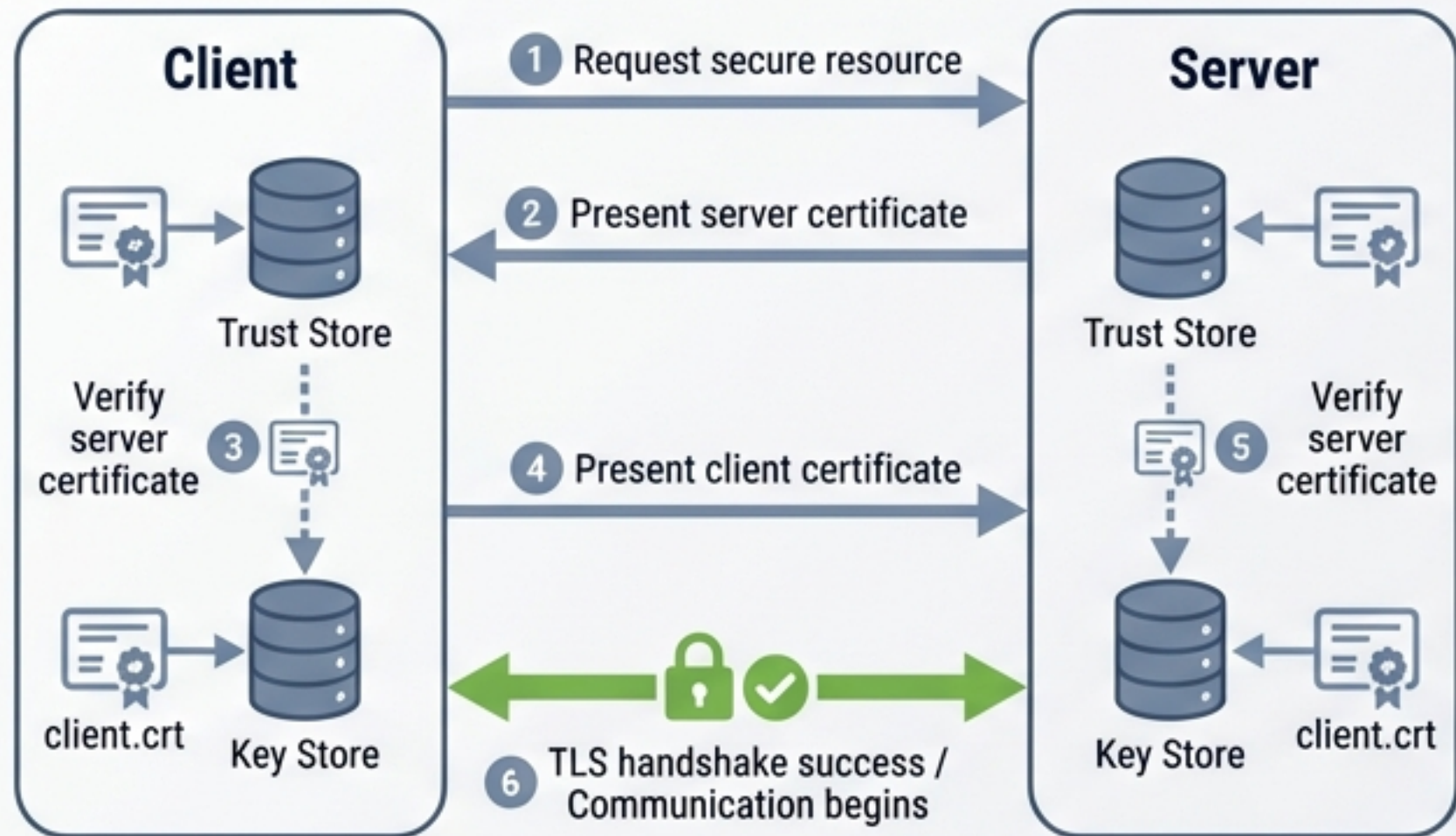
Transport Layer Security (TLS), el sucesor de SSL, garantiza que nadie pueda interceptar o modificar los datos en tránsito (Man-in-the-Middle).



```
server:
  port: 8443
  ssl:
    enabled: true
    key-store: classpath:keystore.p12
    key-store-password: mypassword
    key-store-type: PKCS12 # Estándar desde Java 9
```


Capa 3: Zero-Trust Puro con Mutual TLS (mTLS)

En un entorno Zero-Trust, el servidor tampoco confía en el cliente por defecto. Mutual TLS exige que ambas partes presenten certificados criptográficos válidos. Recomendado para APIs expuestas a terceros o mallas de servicios (Service Mesh).



Para forzar mTLS en Spring Boot (Equivalente al required de Quarkus):

```
server.ssl.client-auth=need
server.ssl.trust-store=classpath:truststore.p12
```


Capa 4: Control de Acceso Basado en Roles (RBAC)

Mapear los claims del JWT (como groups o roles) a privilegios de ejecución en el código.

Quarkus

```
@RolesAllowed("ADMIN")  
public String getAdminsOnly() { ... }
```

Spring Boot

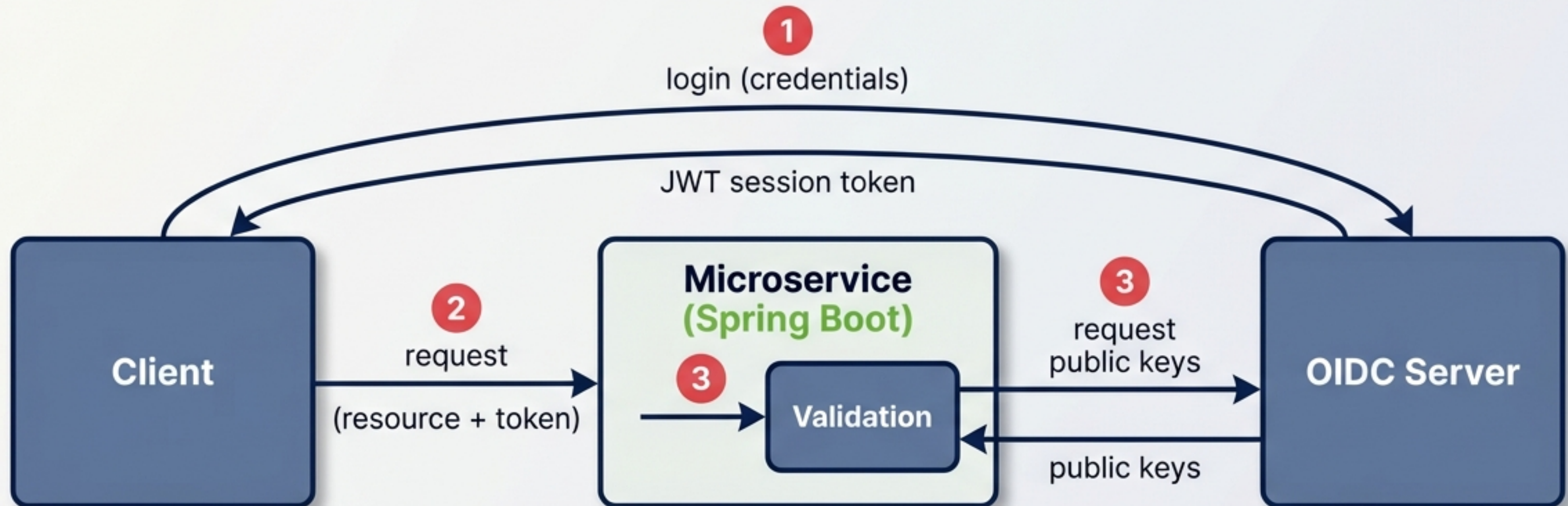
```
@PreAuthorize("hasRole('ADMIN')")  
public String getAdminsOnly() { ... }
```



Nota Técnica: En Spring Boot, requerimos un `JwtAuthenticationConverter` personalizado para transformar los claims del token en `GrantedAuthority` con el prefijo `ROLE_`.

Capa 5: Delegando la Identidad (SSO y OIDC)

Para Single Sign-On (SSO), los microservicios no deben gestionar contraseñas. OpenID Connect (OIDC) delega la autenticación a un servidor dedicado (Identity Provider) como Keycloak.



Integración de Keycloak en Spring Boot

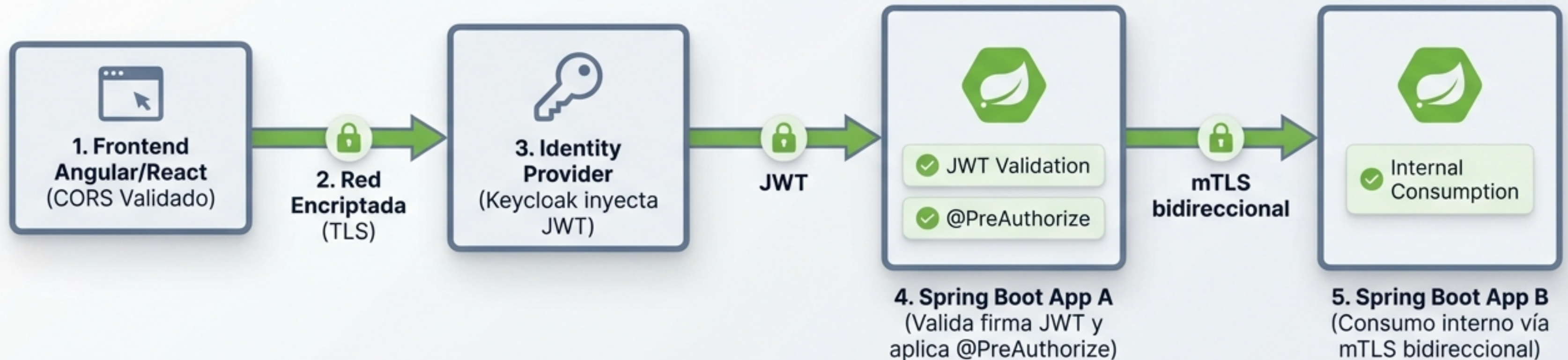
En Spring Boot 3+, la configuración como Resource Server es extremadamente limpia y automática. El framework descarga las llaves públicas (JWK Set) del Identity Provider automáticamente al arrancar.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://[KEYCLOAK_SERVER]/realms/[REALM_NAME]
```



El Ecosistema Completo en Acción

El viaje de una petición 100% segura.



Conclusión: La Seguridad es una Arquitectura

"La seguridad no es un destino al que se llega, es una capa de verificación tras otra."

- ✓ Navegadores controlados (CORS)
- ✓ Comunicaciones impenetrables (TLS / mTLS)
- ✓ Identidades criptográficas (JWT)
- ✓ Autorización estricta (RBAC / @PreAuthorize)
- ✓ Identidad Centralizada (SSO / OIDC con Keycloak)

